HDOC Day-62 Assignment

C PROGRAMMING & DATA STRUCTURES SOLUTION

`Activity #62`

TARGET BATCHES	TOPIC	SUBMISSION STATUS
Batch 15 & Batch 16	Pointers & Two-Pointer Algorithm	Complete & Verified

1. Problem Overview

The objective of this task is to implement a robust C program that reads two sorted variable-length integer arrays, `arr1[]` and `arr2[]`, and merges them into a third sorted array, `merged[]`. All array passing and manipulations between function parameters must strictly utilize **pointers**.

Key Functional Requirements

- **Dynamic Input:** Read sizes and elements for two dynamic variable-length integer arrays sorted in ascending order.
- **Pointer-Based Interface:** Pass array addresses directly to `merge_sorted_arrays()` using pointer arguments.
- **Order & Duplicates Preservation:** Maintain exact ascending order in `merged[]` while keeping all duplicate values intact.
- **Optimal Algorithm:** Implement the standard linear **Two-Pointer Technique** achieving `O(N + M)` time complexity.
- **Formatted Output:** Print the resultant elements of `merged[]` as space-separated integer values.

2. Algorithmic Design & Logic Flow

> **Why the Two-Pointer Technique?**
>
> Sorting an unsourced combined array takes `O((N+M) log(N+M))` time. Since input arrays are pre-sorted, comparing current elements via two pointer offsets allows linear scanning in `O(N + M)` time and `O(1)` auxiliary pointers, meeting optimal efficiency criteria.

3. Input / Output Specification & Test Cases

Test Case	Array 1 (Size, Data)	Array 2 (Size, Data)	Expected Output
Example 1	`N1 = 4` `1 3 5 7`	`N2 = 4` `2 4 6 8`	`1 2 3 4 5 6 7 8`
Example 2	`N1 = 4` `1 2 2 4`	`N2 = 3` `2 3 5`	`1 2 2 2 3 4 5`
Example 3	`N1 = 3` `1 4 7`	`N2 = 5` `2 3 5 6 8`	`1 2 3 4 5 6 7 8`

4. Source Code Implementation

Below is the standard, modular C code adhering strictly to pointer syntax and memory safety standards.

```
/* * HDOC Day-62: Merging Two Sorted Variable-Length Arrays * Technique: Two-Pointer Meth
```

5. Code Structure Explanation

1. **Pointer Dereferencing:** Expression `*(arr1 + i)` is used instead of subscript notation `arr1[i]` to strictly fulfill the pointer requirement.
2. **Memory Management:** `malloc()` dynamically allocates memory on the heap, ensuring support for variable length arrays, followed by explicit cleanup with `free()`.

6. Detailed Execution Trace (Example 2)

Tracing Example 2 execution step-by-step: `arr1 = [1, 2, 2, 4]`, `arr2 = [2, 3, 5]`.

Step	i (arr1)	j (arr2)	Comparison	Action Taken	`merged[]` State
1	0 (Val: 1)	0 (Val: 2)	1 <= 2 (True)	Pick arr1[0] (1), i++	[1]
2	1 (Val: 2)	0 (Val: 2)	2 <= 2 (True)	Pick arr1[1] (2), i++	[1, 2]
3	2 (Val: 2)	0 (Val: 2)	2 <= 2 (True)	Pick arr1[2] (2), i++	[1, 2, 2]
4	3 (Val: 4)	0 (Val: 2)	4 <= 2 (False)	Pick arr2[0] (2), j++	[1, 2, 2, 2]
5	3 (Val: 4)	1 (Val: 3)	4 <= 3 (False)	Pick arr2[1] (3), j++	[1, 2, 2, 2, 3]
6	3 (Val: 4)	2 (Val: 5)	4 <= 5 (True)	Pick arr1[3] (4), i++	[1, 2, 2, 2, 3, 4]
7	4 (End)	2 (Val: 5)	Loop Terminated	Flush remaining arr2[2] (5)	[1, 2, 2, 2, 3, 4, 5]

7. Complexity Analysis

- **Time Complexity — O(N + M):** Each element from `arr1` and `arr2` is visited exactly once during the linear scanning phase. Thus, for arrays of size N and M, the loop executes at most `N + M` iterations.
- **Space Complexity — O(N + M):** Auxiliary space allocated dynamically for the `merged[]` array storing all combined elements.

8. Verification & Conclusion

Solution Integrity Verified

All rules specified in HDOC Day-62 activity (pointer arithmetic, handling duplicates, memory allocation, two-pointer mechanism, space separation) have been strictly satisfied and tested against all given examples.